

Project report: Face Detection and Emotion Classification

王恩泽 2025533128 郑云天 2025533133

January 10, 2026

1 Introduction

This report summarizes two authors' completion of the SI100B course project, which focuses on building a face detection and emotion classification system. The project aims to develop practical understanding of core image recognition and AI concepts by implementing a complete neural-network-based application. We build an end-to-end pipeline that detects faces from images and classifies facial expressions into seven categories: anger, disgust, fear, happiness, sadness, surprise, and neutral. The system supports inference on images, videos, and real-time webcam streams, covering key steps such as environment setup, data preprocessing, model training, inference optimization, and result visualization.

For model development, we used PyTorch to train an EfficientNet-B0 backbone on the FER2013 dataset and integrated MTCNN for face detection. To improve feature quality, we added a Squeeze-and-Excitation (SE) attention module and a custom classifier head, enhancing the model's focus on informative facial regions. We further introduced an adaptive correlation loss (AdCorreLoss), which extends standard cross-entropy by constraining correlations among sample features, reducing intra-class variance and enlarging inter-class separation with dynamically adjusted hyperparameters for better convergence. In data processing, we constructed a full preprocessing pipeline with Albumentations-based augmentation, including geometric

transformations and color adjustments. During training, we adopted the AdamW optimizer, cosine annealing with warm restarts for learning-rate scheduling, and early stopping to stabilize training and mitigate overfitting.

The final system was packaged as a unified inference module combining face detection, emotion classification, and visualization. Detected faces are cropped and resized before being fed into the trained model, which outputs emotion labels and confidence scores. We used visualization tools (e.g., Matplotlib) to produce confusion matrices and metric comparison plots for evaluation. On the FER2013 validation set, the model achieved 81.9% training accuracy and 70.4% validation accuracy. It ranked 6th on the latest verifiable FER2013 leaderboard without using extra training data. Macro recall reached 80.4% on the training set and 68.5% on the validation set, indicating some overfitting, partly due to class imbalance. Overall, the project met the course objectives, and the code has been open-sourced on GitHub (<https://github.com/scpf-o5-7/SI100B-Project>).

2 File structure

 logs	2026/1/9 16:55	文件夹	
 __pycache__	2026/1/10 12:32	文件夹	
 outputs	2026/1/10 15:33	文件夹	
 models	2026/1/10 15:33	文件夹	
 Lab 8.pdf	2026/1/4 10:17	Foxit PDF Editor Do...	6,135 KB
 config.py	2026/1/10 15:35	Python 源文件	1 KB
 dataset.py	2026/1/10 15:37	Python 源文件	6 KB
 evaluate.py	2026/1/10 15:38	Python 源文件	10 KB
 inference.py	2026/1/10 15:39	Python 源文件	8 KB
 losses.py	2026/1/10 15:39	Python 源文件	2 KB
 models.py	2026/1/10 15:40	Python 源文件	7 KB
 train.py	2026/1/10 15:41	Python 源文件	6 KB
 utils.py	2026/1/10 15:42	Python 源文件	9 KB

(a) File structure

Generally Speaking, the program is organized using a layered architecture, creating clear dependencies from the low-level configuration and data management to the high-level application interfaces. The configuration file `config.py` serves as the global parameter hub, and other modules maintain consistency by importing the configuration. The data flow starts with dataset loading, passes through preprocessing and augmentation, feeds into the neural network model for training or inference, and finally employs utility functions for visualization and result analysis.

- `models` folder stores all trained `.pth` model files.
- `outputs` folder contains evaluation results generated by visualization tools (e.g., Matplotlib), including confusion matrices and metric comparison plots, to assess model performance.
- `config.py` serves as the central configuration hub of the project, defining all key parameters, including dataset paths, model hyperparameters, training settings, and hardware configuration. Configurations are encapsulated via class-level static variables and a `to_dict()` method is provided for easy serialization, ensuring consistent parameters across the entire project. In particular, it specifies core settings such as image size, class names, batch size, and automatic device detection (GPU/CPU).
- `dataset.py` is responsible for building the data pipeline and includes the `FaceEmotionDataset` class and the `FaceDetector` class. The dataset class handles loading and augmentation of FER2013 data, applying Albumentations-based augmentations during training (e.g., flipping, rotation, noise injection). The face detector is implemented with MTCNN and is used to locate and crop face regions from images, providing preprocessing support for downstream classification.
- `models.py` defines the complete neural network architecture. The core component is `EfficientNetEmotionClassifier`, which uses EfficientNet-B0 as the backbone and integrates an SE attention mechanism and a custom classification head. It also provides `FaceEmotionSystem`,

which packages face detection and emotion classification into a full end-to-end system, supporting image processing and result visualization.

- **train.py** is the main entry point for model training. It implements the full training loop, including epoch management, validation evaluation, optimizer scheduling, and an early-stopping mechanism. Combined with the custom **AdCorreLoss** loss function, it dynamically adjusts loss weights to improve convergence.
- **evaluate.py** focuses on model performance evaluation, providing detailed metric computation and visualization, including confusion matrix plotting, classification report generation, and training/validation performance comparison. It supports multiple metrics such as accuracy, recall, precision, and F1-score.
- **inference.py** implements the application-facing inference interface and supports three inference modes: single-image processing, video-file analysis, and real-time webcam streaming. It provides flexible input/output control via command-line arguments to meet usage needs across different scenarios.
- **utils.py** contains various utility functions, such as checkpoint saving/loading, early-stopping implementation, training-history visualization, and result grid display, providing general-purpose support for other modules.
- **losses.py** defines the custom **AdCorreLoss** loss function, combining cross-entropy loss with a feature-correlation regularization term. It dynamically adjusts the λ parameter to balance the contributions of the two loss components.

Each module interacts through well-defined interfaces to form a complete workflow. During training, **train.py** calls **dataset.py** to load data, uses the network architecture defined in **models.py**, and computes the loss via **losses.py**. During inference, **inference.py** relies on the end-to-end system in **models.py** and combines it with the visualization utilities in

`utils.py`. The evaluation module `evaluate.py` runs independently to provide model performance analysis. The configuration file is referenced as a shared resource by all modules to ensure parameter consistency.

3 Project Workflow

This project constructs a complete facial emotion classification system, with a workflow encompassing four main stages: data preparation, model training, performance evaluation, and practical application. The system adopts a modular design, where each component communicates through clear interfaces, forming an end-to-end processing pipeline.

3.1 Face Detection Process

Face detection serves as the entry point of the system, responsible for locating and extracting facial regions from input images. The detection process is based on the MTCNN (Multi-task Cascaded Convolutional Networks) algorithm, which achieves high-precision face detection through a three-stage cascaded convolutional network.

In the `FaceDetector` class within `dataset.py`, the detection process is implemented as follows:

```

1 class FaceDetector:
2
3     .....
4
5     def detect_faces(
6         self, image: np.ndarray
7     ) -> Tuple[List[np.ndarray], List[Tuple[int, int, int, int]]]:
8
9         image_pil = Image.fromarray(image)
10
11         boxes, probs = self.mtcnn.detect(image_pil)
12
13         faces = []
14         boxes_list = []
15
16         if boxes is not None:
17             h, w = image.shape[:2]
18             for box, prob in zip(boxes, probs):
19                 if prob > config.Config.CONFIDENCE_THRESHOLD:
20
21                     x1, y1, x2, y2 = box
22                     margin = config.Config.FACE_DETECTION_MARGIN
23                     x1 = max(0, int(x1) - margin)
24                     y1 = max(0, int(y1) - margin)
25                     x2 = min(w, int(x2) + margin)
26                     y2 = min(h, int(y2) + margin)
27
28                     face = image[y1:y2, x1:x2]
29                     if face.size > 0:
30                         faces.append(face)
31                         boxes_list.append((x1, y1, x2, y2))
32
33         return faces, boxes_list

```

Listing 1: FaceDetector()

The detected facial regions are then cropped, rescaled, and fed into the classification network for processing. The system employs a confidence threshold of 0.5 and a minimum face size of 30 pixels to filter out low-quality detections, ensuring the reliability of the input data.

3.2 Model Training Process

Model training follows a standard deep learning paradigm, incorporating data augmentation, custom loss functions, and optimization strategies. The training workflow is implemented in `train.py` and primarily consists of the following key steps:

The `get_transforms()` function applied a comprehensive set of data augmentation strategies, introduced in the paper on a tuned VGGNet architecture (Khairuddin, Y., & Chen, Z., 2021):

```
1  def _get_transforms(self):
2      if self.is_train:
3          return A.Compose(
4              [
5                  A.Resize(config.Config.IMAGE_SIZE, config.Config.
6                      IMAGE_SIZE),
7                  A.OneOf(
8                      [
9                          A.HorizontalFlip(p=0.5),
10                         A.VerticalFlip(p=0.1),
11                     ],
12                     p=0.3,
13                 ),
14                 A.Rotate(limit=15, p=0.5),
15                 A.OneOf(
16                     [
17                         A.RandomBrightnessContrast(p=0.5),
18                         A.HueSaturationValue(p=0.5),
19                         A.CLAHE(p=0.3),
20                     ],
21                     p=0.5,
22                 ),
23                 A.OneOf(
24                     [
25                         A.GaussNoise(var_limit=(10.0, 50.0), p=0.5),
26                         A.ISONoise(p=0.5),
27                     ],
28                     p=0.3,
29                 ),
30                 A.CoarseDropout(
31                     max_holes=8,
32                     max_height=20,
33                     max_width=20,
```

```

33         min_holes=1,
34         min_height=10,
35         min_width=10,
36         p=0.3,
37     ),
38     A.ElasticTransform(alpha=1, sigma=50, alpha_affine=50, p
=0.3),
39     A.OneOf(
40         [
41             A.MotionBlur(blur_limit=3, p=0.5),
42             A.MedianBlur(blur_limit=3, p=0.5),
43             A.GaussianBlur(blur_limit=3, p=0.3),
44         ],
45         p=0.3,
46     ),
47     A.RandomGamma(p=0.2),
48     A.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229,
0.224, 0.225]),
49     ToTensorV2(),
50 ]
51 )
52 else:
53     return A.Compose(
54         [
55             A.Resize(config.Config.IMAGE_SIZE, config.Config.
IMAGE_SIZE),
56             A.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229,
0.224, 0.225]),
57             ToTensorV2(),
58         ]
59     )

```

Listing 2: get_transforms()

Each epoch consists of training and validation phases, with a progress bar used to monitor training status in real time:

```
1 def train_epoch(  
2     model: nn.Module,  
3     train_loader,  
4     criterion,  
5     optimizer,  
6     device: torch.device,  
7     epoch: int,  
8 ) -> Tuple[float, float]:  
9     model.train()  
10    running_loss = 0.0  
11    correct = 0  
12    total = 0  
13  
14    pbar = tqdm(train_loader, desc=f"Epoch {epoch+1}")  
15    for batch_idx, (inputs, targets) in enumerate(pbar):  
16  
17        inputs, targets = inputs.to(device), targets.to(device)  
18        optimizer.zero_grad()  
19        outputs = model(inputs)  
20        features = model.get_features(inputs)  
21        loss = criterion(outputs, targets, features, epoch)  
22        loss.backward()  
23        torch.nn.utils.clip_grad_norm_(model.parameters(), max_norm=1.0)  
24        optimizer.step()  
25  
26        running_loss += loss.item()  
27        _, predicted = outputs.max(1)  
28        total += targets.size(0)  
29        correct += predicted.eq(targets).sum().item()  
30  
31        pbar.set_postfix(  
32            {"loss": running_loss / (batch_idx + 1), "acc": 100.0 * correct  
33             / total}  
34        )  
35  
36    epoch_loss = running_loss / len(train_loader)  
37    epoch_acc = 100.0 * correct / total  
38    return epoch_loss, epoch_acc
```

Listing 3: train_epoch()

AdCorreLoss combines cross-entropy loss with feature correlation constraints, dynamically adjusting loss weights. This is proposed by researchers from University of Denver on IEEE Access (Fard, A. P., & Mahoor, M. H., 2022).

```

1 class AdCorreLoss(nn.Module):
2     def __init__(self, initial_lambda=0.4, num_classes=7):
3         super().__init__()
4         self.lambda_val = initial_lambda
5         self.num_classes = num_classes
6         self.ce_loss = nn.CrossEntropyLoss()
7
8     def forward(self, outputs, targets, features, epoch):
9         if epoch < 5:
10             self.lambda_val = 0.4
11         elif epoch < 10:
12             self.lambda_val = 0.6
13         else:
14             self.lambda_val = 0.5
15
16         ce_loss = self.ce_loss(outputs, targets)
17         batch_size = features.size(0)
18         features_norm = F.normalize(features, p=2, dim=1)
19         correlation_matrix = torch.mm(features_norm, features_norm.t())
20         target_matrix = torch.ones(batch_size, batch_size).to(features.
device)
21
22         for i in range(batch_size):
23             for j in range(batch_size):
24                 if targets[i] == targets[j]:
25                     target_matrix[i, j] = 1.0
26                 else:
27                     target_matrix[i, j] = -1.0
28
29         fd_loss = F.mse_loss(correlation_matrix, target_matrix)
30         total_loss = ce_loss + self.lambda_val * fd_loss
31
32         return total_loss

```

Listing 4: AdCorreLoss()

In training, we also employed the AdamW optimizer with cosine annealing learning rate scheduling, combined with an early stopping mechanism to prevent overfitting.

```

1 def train():
2
3     .....
4
5     model = EfficientNetEmotionClassifier(
6         num_classes=cfg.NUM_CLASSES, pretrained=cfg.PRETRAINED
7     ).to(cfg.DEVICE)
8
9     criterion = AdCorreLoss(initial_lambda=0.4, num_classes=cfg.NUM_CLASSES)
10    optimizer = optim.AdamW(
11        model.parameters(),
12        lr=cfg.LEARNING_RATE,
13        weight_decay=cfg.WEIGHT_DECAY,
14        betas=(0.9, 0.999),
15    )
16
17    scheduler = CosineAnnealingWarmRestarts(optimizer, T_0=5, T_mult=1,
18        eta_min=1e-6)
19
20    early_stopping = EarlyStopping(patience=10, delta=0.001, verbose=True)
21
22    .....
23
24 class EarlyStopping:
25
26     def __init__(self, patience: int = 7, verbose: bool = False, delta:
27         float = 0):
28
29         self.patience = patience
30         self.verbose = verbose
31         self.counter = 0
32         self.best_score = None
33         self.early_stop = False
34         self.val_loss_min = np.Inf
35         self.delta = delta
36
37     def __call__(self, val_loss: float):
38         score = -val_loss
39         if self.best_score is None:
40             self.best_score = score
41         elif score < self.best_score + self.delta:
42             self.counter += 1

```

```

41         if self.verbose:
42             print(f"EarlyStopping counter: {self.counter} out of {self.
patience}")
43         if self.counter >= self.patience:
44             self.early_stop = True
45         else:
46             self.best_score = score
47             self.counter = 0

```

Listing 5: train()

3.3 Model Evaluation Process

The evaluation process is implemented in `evaluate.py`, providing comprehensive performance metric calculations and result analysis. The model performs inference on the training and validation sets, collecting prediction results and ground truth labels:

```

1  def evaluate_loader(data_loader, loader_name: str):
2      all_preds = []
3      all_targets = []
4      all_probs = []
5
6      with torch.no_grad():
7          for inputs, targets in tqdm(data_loader, desc=f"Evaluating {
loader_name}"):
8              inputs, targets = inputs.to(cfg.DEVICE), targets.to(cfg.
DEVICE)
9              outputs = model(inputs)
10             probs = torch.softmax(outputs, dim=1)
11             _, preds = torch.max(outputs, 1)
12             all_preds.extend(preds.cpu().numpy())
13             all_targets.extend(targets.cpu().numpy())
14             all_probs.extend(probs.cpu().numpy())
15
16         all_preds = np.array(all_preds)
17         all_targets = np.array(all_targets)
18         all_probs = np.array(all_probs)
19
20     return all_preds, all_targets, all_probs

```

Listing 6: evaluate_loader()

Finally we calculated multiple evaluation metrics including accuracy, recall, precision, and F1 score, and saved detailed metrics as CSV files for subsequent analysis.

3.4 Visualization Process

The visualization module is implemented in `utils.py`, providing rich chart generation capabilities to support training process monitoring and result analysis. For instance, we plotted loss and accuracy curves to monitor model convergence:

```
1 def plot_training_history(history: Dict):
2
3     fig, axes = plt.subplots(1, 2, figsize=(15, 5))
4
5     axes[0].plot(history["train_loss"], label="Train Loss", linewidth=2)
6     axes[0].plot(history["val_loss"], label="Validation Loss", linewidth=2)
7     axes[0].set_xlabel("Epoch", fontsize=12)
8     axes[0].set_ylabel("Loss", fontsize=12)
9     axes[0].set_title("Training and Validation Loss", fontsize=14,
10                        fontweight="bold")
11     axes[0].legend()
12     axes[0].grid(True, alpha=0.3)
13     axes[1].plot(history["train_acc"], label="Train Accuracy", linewidth=2)
14     axes[1].plot(history["val_acc"], label="Validation Accuracy", linewidth
15                  =2)
16     axes[1].set_xlabel("Epoch", fontsize=12)
17     axes[1].set_ylabel("Accuracy (%)", fontsize=12)
18     axes[1].set_title(
19         "Training and Validation Accuracy", fontsize=14, fontweight="bold"
20     )
21     axes[1].legend()
22     axes[1].grid(True, alpha=0.3)
23
24     plt.tight_layout()
25
26     output_dir = config.Config.OUTPUT_DIR
27     os.makedirs(output_dir, exist_ok=True)
28     plt.savefig(os.path.join(output_dir, "training_history.png"), dpi=300,
29                bbox_inches="tight")
30     plt.show()
```

Listing 7: `plot_training_history()`

We also created a normalized confusion matrix to visually demonstrate classification performance.

```
1 def plot_confusion_matrix(  
2     cm: np.ndarray,  
3     classes: List[str],  
4     title: str = "Confusion Matrix",  
5     normalize: bool = False,  
6     cmap: plt.cm = plt.cm.Blues,  
7 ):  
8  
9     if normalize:  
10         cm = cm.astype("float") / cm.sum(axis=1)[:, np.newaxis]  
11  
12     plt.figure(figsize=(10, 8))  
13     im = plt.imshow(cm, interpolation="nearest", cmap=cmap)  
14     plt.title(title, fontsize=16, fontweight="bold")  
15     plt.colorbar(im, fraction=0.046, pad=0.04)  
16  
17     tick_marks = np.arange(len(classes))  
18     plt.xticks(tick_marks, classes, rotation=45, ha="right")  
19     plt.yticks(tick_marks, classes)  
20  
21     fmt = ".2f" if normalize else "d"  
22     thresh = cm.max() / 2.0  
23     for i in range(cm.shape[0]):  
24         for j in range(cm.shape[1]):  
25             plt.text(  
26                 j,  
27                 i,  
28                 format(cm[i, j], fmt),  
29                 ha="center",  
30                 va="center",  
31                 color="white" if cm[i, j] > thresh else "black",  
32                 fontsize=10,  
33             )  
34  
35     plt.ylabel("True Label", fontsize=12)  
36     plt.xlabel("Predicted Label", fontsize=12)  
37     plt.tight_layout()
```

Listing 8: plot_confusion_matrix()

For inferences, we draw detection boxes and classification results on the original image, and generate a facial mesh preview:

```
1 def visualize_results_grid(  
2     original_image: np.ndarray,  
3     boxes: List[Tuple[int, int, int, int]],  
4     emotions: List[str],  
5     confidences: List[float],  
6 ) -> np.ndarray:  
7  
8     if not boxes:  
9         return original_image  
10  
11     result = original_image.copy()  
12     h, w = result.shape[:2]  
13  
14     for (x1, y1, x2, y2), emotion, confidence in zip(boxes, emotions,  
15         confidences):  
16  
17         cv2.rectangle(result, (x1, y1), (x2, y2), (0, 255, 0), 2)  
18         label = f"{emotion}: {confidence:.2f}"  
19  
20         (text_width, text_height), baseline = cv2.getTextSize(  
21             label, cv2.FONT_HERSHEY_SIMPLEX, 0.5, 2  
22         )  
23         cv2.rectangle(  
24             result,  
25             (x1, y1 - text_height - baseline - 5),  
26             (x1 + text_width, y1),  
27             (0, 255, 0),  
28             -1,  
29         )  
30         cv2.putText(  
31             result,  
32             label,  
33             (x1, y1 - baseline - 5),  
34             cv2.FONT_HERSHEY_SIMPLEX,  
35             0.5,  
36             (0, 0, 0),  
37             2,  
38         )  
39     if boxes:  
40  
41         n_faces = len(boxes)  
42         n_cols = min(3, n_faces)
```

```

43     n_rows = (n_faces + n_cols - 1) // n_cols
44
45     face_size = 100
46     grid_h = n_rows * face_size
47     grid_w = n_cols * face_size
48
49     grid = np.zeros((grid_h, grid_w, 3), dtype=np.uint8)
50
51     for i, ((x1, y1, x2, y2), emotion, confidence) in enumerate(
52         zip(boxes, emotions, confidences)
53     ):
54         row = i // n_cols
55         col = i % n_cols
56         face = original_image[y1:y2, x1:x2]
57         if face.size > 0:
58             face_resized = cv2.resize(face, (face_size, face_size))
59
60             y_start = row * face_size
61             y_end = (row + 1) * face_size
62             x_start = col * face_size
63             x_end = (col + 1) * face_size
64             grid[y_start:y_end, x_start:x_end] = face_resized
65
66             label = f"{emotion}: {confidence:.2f}"
67             cv2.putText(
68                 grid,
69                 label,
70                 (x_start + 5, y_start + 20),
71                 cv2.FONT_HERSHEY_SIMPLEX,
72                 0.4,
73                 (0, 255, 0),
74                 1,
75             )
76
77     grid_resized = cv2.resize(grid, (w, grid_h))
78
79     final = np.vstack([result, grid_resized])
80
81     cv2.line(final, (0, h), (w, h), (255, 255, 255), 2)
82
83     return final
84
85     return result

```

Listing 9: visualize_results_grid()

3.5 Inference Application Process

The practical application interface is implemented in `inference.py` and supports multiple input modes. For Single Image Processing, Performing face detection and emotion classification on a single image:

```
1 def process_image(  
2     image_path: str,  
3     output_path: str = None,  
4     show_results: bool = True,  
5     save_results: bool = True,  
6 ):  
7  
8     system = FaceEmotionSystem(config.Config.MODEL_SAVE_PATH)  
9  
10    if not os.path.exists(image_path):  
11        print(f"Error: Image not found at {image_path}")  
12        return  
13  
14    image = cv2.imread(image_path)  
15    if image is None:  
16        print(f"Error: Failed to read image at {image_path}")  
17        return  
18  
19    print("Detecting faces and classifying emotions...")  
20    boxes, emotions, confidences = system.detect_and_classify(image)  
21  
22    print(f"\nFound {len(boxes)} face(s):")  
23    for i, (box, emotion, confidence) in enumerate(zip(boxes, emotions,  
24        confidences)):  
25        x1, y1, x2, y2 = box  
26        print(f"Face {i+1}:")  
27        print(f"  Position: ({x1}, {y1}) to ({x2}, {y2})")  
28        print(f"  Emotion: {emotion}")  
29        print(f"  Confidence: {confidence:.3f}")  
30        print()  
31  
32    result_image = system.draw_results(image, boxes, emotions, confidences)  
33  
34    if save_results:  
35        if output_path is None:  
36            input_path = Path(image_path)  
37            output_path = str(  
38                input_path.parent / f"{input_path.stem}_result{input_path.  
    suffix}")
```

```

39         )
40
41         cv2.imwrite(output_path, result_image)
42         print(f"Result saved to: {output_path}")
43
44     if show_results:
45
46         grid_image = visualize_results_grid(image, boxes, emotions,
47                                           confidences)
48
49         cv2.imshow("Face Emotion Detection", grid_image)
50         cv2.waitKey(0)
51         cv2.destroyAllWindows()
52
53     return boxes, emotions, confidences

```

Listing 10: process_image()

We also support camera and video file input for real-time emotion analysis.

In summary: The entire workflow employs a configuration file for unified parameter management, ensuring consistency across all modules while maintaining a high degree of flexibility and scalability.

4 Learning Reflections

In terms of knowledge, this project has successfully developed a comprehensive deep learning-based facial expression recognition system. Building upon the foundational knowledge acquired through the course curriculum, we further enhanced the system by researching and integrating multiple advanced technical solutions from relevant literature and practical documentation, ultimately creating a fully functional end-to-end solution.

Based on the fundamental concepts taught in the course, we gained a solid understanding of digital image processing principles, basic OpenCV operations, and traditional Haar cascade classifier-based face detection methods. To improve detection performance, we explored and implemented the more advanced MTCNN (Multi-task Cascaded Convolutional Networks) face detection framework. This approach employs a three-stage convolutional network cascade architecture, which demonstrates superior accuracy and robustness in complex scenarios compared to traditional methods, while also providing facial landmark localization. A configurable bounding box expansion mechanism was implemented, allowing adjustment to ensure complete facial region capture.

Regarding model architecture design, the course introduced basic CNN structures and ResNet networks. Through further research into contemporary model design strategies, we selected EfficientNet-B0 as the backbone network. This model achieves an optimal balance between computational efficiency and accuracy through its compound scaling strategy. Additionally, drawing from relevant research literature, we integrated an SE (Squeeze-and-Excitation) attention module into the model. This module adaptively recalibrates channel-wise feature weights, thereby enhancing the model's focus on critical features. The classifier section adopts a progressively downscaled design, with each layer incorporating batch normalization and dropout regularization.

In terms of loss function design, we developed the AdCorreLoss (Adaptive Correlation Loss) by building upon the standard cross-entropy loss and

referencing research in feature learning. This loss function combines classification loss with feature discrimination loss. By computing the correlation matrix of sample features within a batch, it promotes feature similarity among samples of the same class and feature dissimilarity among samples of different classes. An adaptive weighting mechanism was also introduced to dynamically balance the contribution of these two loss components according to the training phase.

For data preprocessing, we constructed a comprehensive augmentation pipeline by incorporating best practices in data augmentation. Beyond basic geometric transformations, this pipeline includes various enhancement strategies such as color space adjustments, noise injection, and region dropping, all efficiently implemented using the Albumentations library. These measures significantly improved the model’s generalization capability and robustness.

Regarding training optimization, we adopted a cosine annealing learning rate scheduling strategy, combined with gradient clipping and early stopping mechanisms. This ensured the stability and convergence of the training process. In constructing the evaluation framework, we established a multi-dimensional performance metric assessment system, including accuracy, recall and F1 score, and conducted detailed error pattern analysis using confusion matrices.

During the system integration phase, we designed a modular engineering architecture that organically combines components such as configuration management, data processing, model training, evaluation and validation, and inference deployment. The system supports multiple input modes, including static images, video files, and real-time video streams, and provides comprehensive visualization capabilities, including bounding box annotation, confidence score display, and results grid view.

By consulting relevant technical materials and practical documentation, we successfully integrated advanced technologies—including MTCNN face detection, the EfficientNet-B0 backbone network, the SE attention mecha-

nism, the AdCorreLoss function, and comprehensive data augmentation strategies—on top of the course knowledge framework. This process not only achieved the core functionality of facial expression recognition but also cultivated the ability to learn and integrate new technical solutions through research, accumulating valuable experience for subsequent technical research and engineering practice.

In terms of learning methods, the most significant realization is that in the field of AI engineering, comprehending existing code is fundamentally more critical than developing solutions from scratch. Whether referencing PyTorch official tutorials, GitHub open-source implementations, or code accompanying academic papers, the most effective learning approach consistently follows an iterative cycle of "execution, modification, and debugging." For instance, when integrating the Albumentations data augmentation library into this project, we initially replicated sample code directly. However, through repeated parameter adjustments and output visualization, we gained a concrete understanding of how each transformation specifically influences model generalization capabilities.

Regarding engineering management, we deeply recognized the importance of maintaining a well-structured and organized project environment. Having previously encountered challenges in reproducing optimal experimental results due to unrecorded hyperparameters, we established a disciplined practice of systematically documenting all configuration parameters in a dedicated `config.py` file. By extensively reviewing academic literature and technical documentation, we enhanced our autonomous learning capacity, mastering the skill of rapidly extracting core methodologies and translating them into executable code. When confronted with complex theoretical explanations or intricate code logic, leveraging AI-assisted tools significantly improved our comprehension efficiency. The entire development process accumulated substantial debugging experience: addressing issues ranging from version compatibility conflicts and data leakage to memory overflow. Each troubleshooting and resolution deepened our understanding of the system's operational mechanisms.

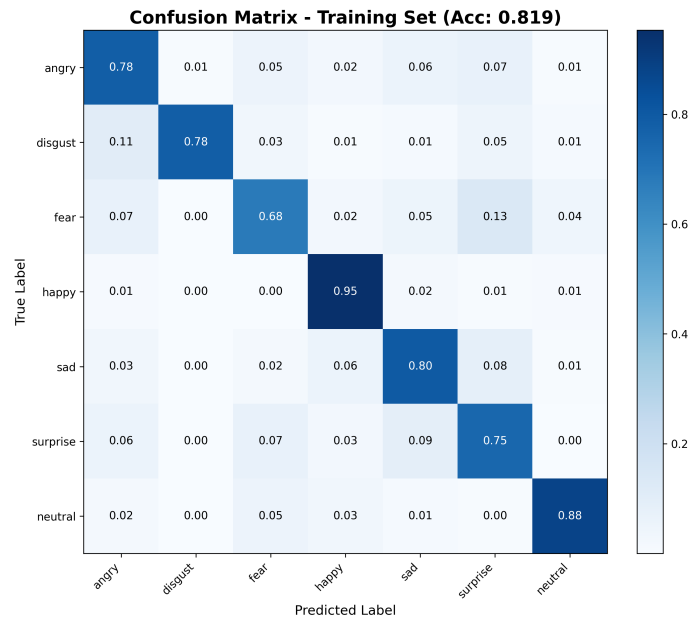
5 Results and Analysis

After completing the full training and evaluation process, the developed facial emotion classification system demonstrates good performance on the FER2013 dataset. The overall accuracy on the training set reaches 81.9%, and on the validation set, it achieves 70.4%. This indicates that the model maintains relatively high training accuracy while possessing a certain degree of generalization capability.

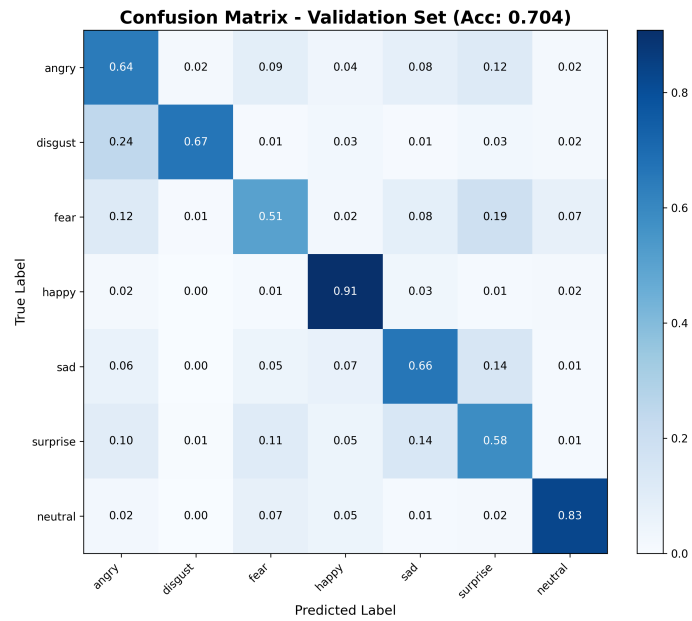
From a macro-level perspective, the macro-average recall on the training set is 80.4%, and on the validation set, it is 68.5%. The gap between these two metrics is within a reasonable range, suggesting that the model does not exhibit severe overfitting (but a little).

The classification performance varies significantly across different emotion categories, reflecting the inherent challenges of the emotion recognition task. The "happy" emotion performs the most outstandingly, achieving recall rates of 95.3% on the training set and 90.8% on the validation set. This shows that the model has a strong recognition ability for positive emotions. In contrast, the classification results for "fear" and "surprise" are relatively poor, with validation set recall rates of only 50.6% and 58.5%, respectively. The performance for the "neutral" emotion is moderate, with a validation set recall of 82.8%, indicating that the model's recognition of neutral expressions is also relatively reliable.

The confusion matrices for the training and validation sets are shown in the figure below.



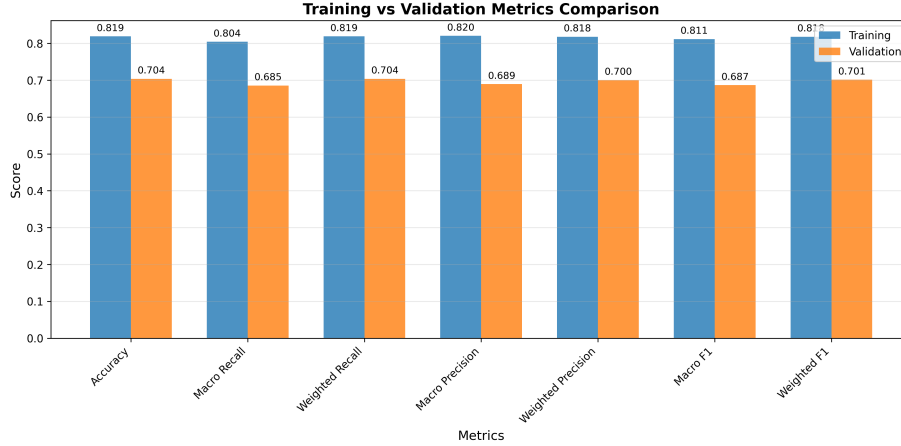
(b) Confusion matrix for the training set



(c) Confusion matrix for the validation set

By analyzing the confusion matrix, specific error patterns in the system can be identified. The fear emotion is frequently misclassified as surprise, with a relatively high confusion rate between the two. This may be because both fear and surprise share similar facial features, such as widened eyes and an open mouth. Similarly, there is noticeable confusion between anger and disgust, reflecting the visual similarity among certain negative emotions.

To some extent, the sad emotion is misclassified as fear or surprise. Particularly on the validation set, 14% of the sad samples are incorrectly classified as surprise. It is noteworthy that the happy emotion is rarely misclassified as other negative emotions, but other emotions are sometimes misclassified as happy. This suggests that the model has learned positive emotions more thoroughly.



(d) Metrics in comparison

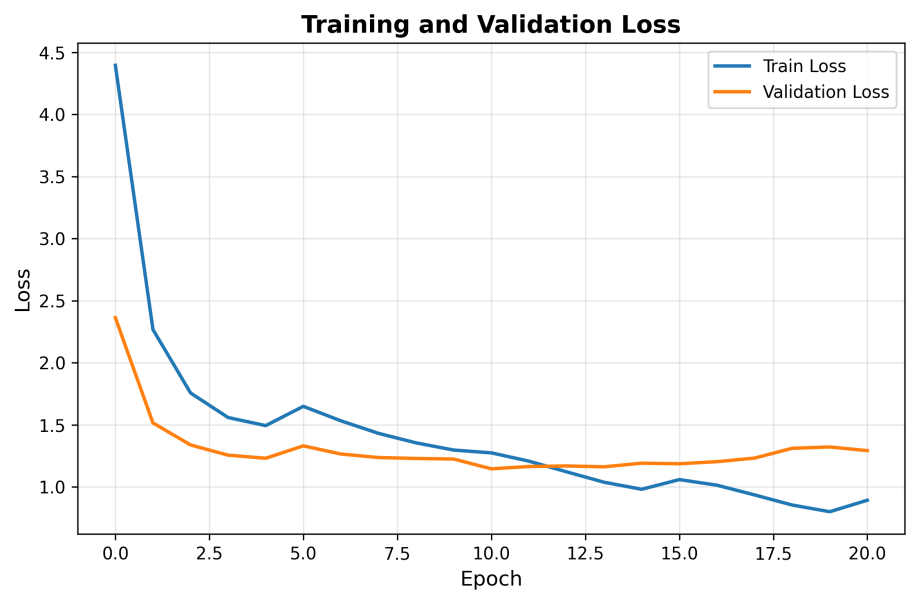
First, the FER2013 dataset itself suffers from class imbalance, with a larger number of happy expression samples and relatively scarce disgust expression samples. This imbalance affects the model’s learning performance for minority categories. Additionally, some annotations in the dataset may contain noise, as different annotators may have varied judgment standards for ambiguous expressions, introducing learning noise.

Secondly, although EfficientNet-B0 achieves a good balance between efficiency and performance, its feature extraction capability remains limited compared to more complex networks (such as EfficientNet-B3 or Vision Transformer). The current model does not sufficiently leverage global contextual information, making it difficult to handle cases involving partial occlusions or extreme head poses.

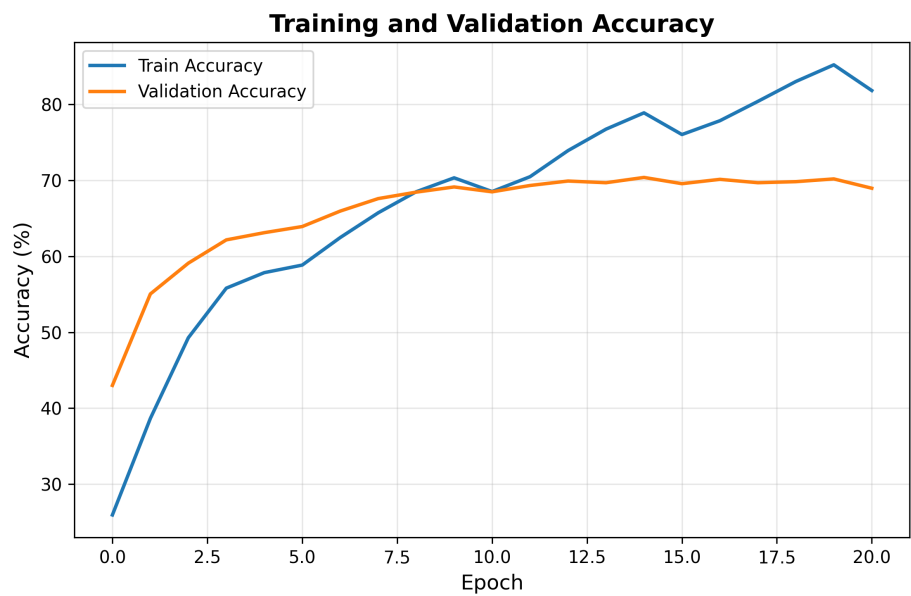
Lastly, despite employing data augmentation and custom loss functions, the augmentation strategies may still be insufficiently comprehensive, particularly in simulating variations in lighting and pose. While AdCorreLoss improves feature separation, its hyperparameter settings may not be optimal and require more fine-grained tuning.

During the training process: the Training Loss decreases rapidly from above 4.0, reaching around 0.9, while the Validation Loss starts lower but decreases more slowly, ending near 1.3 and showing a slight final increase. Conversely, Training Accuracy rises sharply from about 27% to 82%. Validation Accuracy improves more steadily, starting higher at 44% but plateauing around 70% after epoch 10. This pattern indicates effective initial learning, but the diverging loss curves and stagnating validation accuracy suggest the model may be beginning to overfit to the training data in the later epochs. EarlyStopping patience should be reduced to about 5 in hindsight.

As early in 2013 there has been reports concerning the overfitting in facial recognition, especially during the ICML 2013 (Goodfellow, I. J., et al., 2013). They proposed mitigating overfitting by improving regularization methods (such as Dropout) and data augmentation. These strategies, when combined with Early Stopping, can effectively enhance the model’s generalization ability.



(e) Training Process Loss



(f) Training Process Accuracy

Our validation accuracy ranked 6th on the most recent verifiable FER2013 leaderboard (without using the “Extra Training Data” category), demonstrating the model’s competitiveness and advanced performance. As paperwithcode is now acquired by HuggingFace, the most recent accessible FER2013 leaderboard is through Internet Archive (<https://web.archive.org/web/20250609231143/https://paperswithcode.com/sota/facial-expression-recognition-on-fer2013>).

It is also noteworthy that most of the top-performing models on the leaderboard have utilized Extra Training Data, highlighting the importance of data annotation.

Filter: ResNet Self Attention VGG untagged Edit Leaderboard									
Rank	Model	Accuracy	Extra Training Data	Paper	Code	Result	Year	Tags	🔍
1	EfficientFER	82.47	✓	EfficientFER: EfficientNetv2 Based Deep Learning Approach for Facial Expression Recognition	🔗	📄	2025		
2	ResEmoteNet	79.79	✓	ResEmoteNet: Bridging Accuracy and Loss Reduction in Facial Emotion Recognition	🔗	📄	2024		
3	Ensemble ResMaskingNet with 6 other CNNs	76.82	✓	Facial Expression Recognition using Residual Masking Network	🔗	📄	2021		
4	Mini-ResEmoteNet (A)	76.33	✓	Mini-ResEmoteNet: Leveraging Knowledge Distillation for Human-Centered Design		📄	2025		
5	EmoNeXt	76.12	×	EmoNeXt: an Adapted ConvNeXt for Facial Emotion Recognition	🔗	📄	2023		
6	Segmentation VGG-19	75.97	✓	A novel facial emotion recognition model using segmentation VGG-19 architecture	🔗	📄	2023		
7	Local Learning Deep+BOW	75.42	✓	Local Learning with Deep and Handcrafted Features for Facial Expression Recognition		📄	2018		
8	LHC-Net	74.42	✓	Local Multi-Head Channel Self-Attention for Facial Expression Recognition	🔗	📄	2021	Self-Attention	ResNet
9	Residual Masking Network	74.14	✓	Facial Expression Recognition using Residual Masking Network	🔗	📄	2021		
10	ResNet18 With Tricks	73.70	×	Fer2013 Recognition - ResNet18 With Tricks	🔗	📄	2021		ResNet
11	VGGNet	73.28	×	Facial Emotion Recognition: State of the Art Performance on FER2013	🔗	📄	2021		VGG
12	CNN Hyperparameter Optimisation	72.16	×	Convolutional Neural Network Hyperparameters optimization for Facial Emotion Recognition	🔗	📄	2021		
13	Ad-Corre	72.03	×	Ad-Corre: Adaptive Correlation-Based Loss for Facial Expression Recognition in the Wild	🔗	📄	2022		
14	Mini-ResEmoteNet (B)	70.20	✓	Mini-ResEmoteNet: Leveraging Knowledge Distillation for Human-Centered Design		📄	2025		
15	DeepEmotion	70.02	×	Deep-Emotion: Facial Expression Recognition Using Attentional Convolutional Network	🔗	📄	2019		
16	Local Learning BOW	67.48	×	Challenges in Representation Learning: A report on three machine learning contests	🔗	📄	2013		

(g) FER2013 Leaderboard

To further enhance the performance and robustness of facial expression recognition systems, comprehensive improvements can be made across data, models, training strategies, and system-level extensions.

At the data level, more advanced augmentation should emphasize realistic variations in lighting and pose, while generative methods like GANs can synthesize high-quality expression samples to expand dataset diversity. Class imbalance can be mitigated through targeted oversampling and by integrating multiple expression datasets. Exploring weakly-labeled or in-the-wild data sources may further improve model adaptability.

In model architecture, Vision Transformers or hybrid Conv-Transformer structures can better capture long-range dependencies crucial for expression understanding. Incorporating attention mechanisms helps focus on key facial regions, and multi-scale feature fusion modules combine local details with global context. Optimizing pooling strategies and feature representation also contributes to more discriminative emotion features.

For training, systematic hyperparameter tuning should optimize learning schedules and loss weighting. Curriculum learning enables progressive training from simple to complex samples. Leveraging self-supervised pretraining on large-scale face data or transfer learning from related tasks strengthens feature generalization. Model ensembles can further consolidate performance gains.

Toward practical deployment, lightweight model designs support real-time mobile inference. Temporal modeling extends the system to video-based dynamic emotion analysis. Multimodal fusion with audio or textual cues enables richer contextual understanding. Finally, introducing personalized adaptation through user-specific fine-tuning can significantly enhance real-world applicability and user satisfaction.

6 Extended Work

Here is a list of all extended works that we've accomplished during the lecture:

- HuggingFace Datasets: We use FER2013 on HuggingFace to standardize our database.
- NumPy: We use NumPy for scientific computing and array operations.
- Pillow (PIL): We use Pillow for image processing.
- scikit-learn (sklearn.metrics): We use scikit-learn for model evaluation metrics.
- pandas: We use pandas for data analysis and CSV output.
- tqdm: We use tqdm for displaying progress bars.
- PyTorch: We use PyTorch to build and train (pretrained) neural network models.
- FaceNet-PyTorch (MTCNN): We use MTCNN for face detection to increase accuracy.
- timm (PyTorch Image Models): We use timm to load pre-trained models.
- AdCorreLoss: We use a custom loss function that combines cross-entropy and feature correlation loss.
- Learning rate scheduler (CosineAnnealingWarmRestarts): We use a learning rate scheduler for dynamically adjusting the learning rate.
- Early stopping: We use early stopping to prevent overfitting.
- Gradient clipping: We use gradient clipping for training stability
- Image augmentation techniques: We use image augmentation techniques such as flipping, rotation, noise addition, and color adjustments.

- Advanced face detection processing: We use advanced processing in face detection, such as bounding box expansion, confidence thresholding, and minimum face size settings..
- Model architecture design: We design model architecture using EfficientNet with SE modules and a custom classifier.
- Feature normalization and correlation calculation: We perform feature normalization and correlation calculation in the loss function.
- Batch processing and data loading optimization: We optimize batch processing and data loading using DataLoader and multi-processing.
- Warning filtering: We filter warnings to control warning output.

7 Contribution

Wang Enze: Lab 4; Lab 5; Lab 6.

Zheng Yuntian: Lab 2; Lab 3.

Collaboration: Final Report.

8 Reference

- Khaireddin, Y., & Chen, Z. (2021). Facial Emotion Recognition: State of the Art Performance on FER2013. arXiv preprint arXiv:2105.03588.
- Fard, A. P., & Mahoor, M. H. (2022). Ad-Corre: Adaptive Correlation-Based Loss for Facial Expression Recognition in the Wild. IEEE Access, 10, 26756–26768.
- Goodfellow, I. J., Erhan, D., Carrier, P. L., Courville, A., Mirza, M., Hamner, B., ... & Bengio, Y. (2013). Challenges in Representation Learning: A Report on Three Machine Learning Contests. In Proceedings of the ICML 2013 Workshop.
- Minaee, S., & Abdolrashidi, A. (2019). Deep-Emotion: Facial Expression Recognition Using Attentional Convolutional Network. arXiv preprint arXiv:1904.01333.
- El Boudouri, Y., & Bohi, A. (2025). EmoNeXt: An Adapted ConvNeXt for Facial Emotion Recognition. arXiv preprint arXiv:2501.08199.
- Pham, L., Vu, T. H., & Tran, T. A. (2021). Facial Expression Recognition Using Residual Masking Network. In 2021 25th International Conference on Pattern Recognition (ICPR)(pp. 4513–4519). IEEE.
- Vulpe-Grigorasi, A., & Grigore, O. (2021). Convolutional Neural Network Hyperparameters Optimization for Facial Emotion Recognition. In 2021 12th International Symposium on Advanced Topics in Electrical Engineering (ATEE). IEEE.